LARAVEL FROM SCRATCH

COMPLETE STUDY GUIDE — 2026 EDITION

43 Episodes · Section 1 through Final Project

Based on Jeffrey Way's Laracasts course
PHP 8.4 · Laravel 12 · SQLite · Herd

LARAVEL CHEAT SHEET

Quick reference for the patterns, commands, and concepts you'll reach for constantly. Bookmark this — you'll come back to it throughout the course.

DIRECTORY STRUCTURE

```
app/
  Http/
    Controllers/        # Request handlers (IdeaController, UserController)
    Middleware/         # Filters that run before/after routes
    Requests/           # Form Request validation classes
  Models/               # Eloquent models (Idea.php, User.php)
  Policies/             # Authorization policies
  Providers/            # Service providers (AppServiceProvider)
  Actions/              # Single-purpose action classes (CreateIdeaAction)

bootstrap/
  app.php               # App configuration + middleware + routing

config/                 # Environment-agnostic config files
database/
  factories/            # Model factories for testing/seeding
  migrations/           # Schema changes over time
  seeders/              # Seed data

public/
  index.php             # Single entry point — everything flows through here

resources/
  views/                # Blade templates
  js/                   # JavaScript entry points (Alpine, Vite)
  css/                  # Tailwind entry point

routes/
  web.php               # Web routes (sessions, CSRF)
  console.php           # Artisan command routes
  api.php               # Stateless API routes (if using)

storage/
  app/                  # User-uploaded files
  framework/            # Compiled Blade, sessions, cache
  logs/                 # laravel.log

tests/
  Feature/              # Full HTTP request tests
  Unit/                 # Isolated class tests

.env                    # Environment variables (NEVER commit)
artisan                 # CLI entry point
composer.json           # PHP dependencies
vite.config.js          # Asset bundler config
```

ESSENTIAL ARTISAN COMMANDS

Command	Purpose
`php artisan serve`	Start dev server (Herd auto-serves .test domains)
`php artisan tinker`	Interactive REPL with full app loaded
`php artisan make:model Idea -mcr`	Model + migration + controller (resource)
`php artisan make:migration create_ideas_table`	New migration file
`php artisan migrate`	Run pending migrations
`php artisan migrate:refresh --seed`	Drop all, re-run migrations, re-seed
`php artisan make:controller IdeaController --resource`	Resource controller (7 REST methods)
`php artisan make:request StoreIdeaRequest`	Form Request validation class
`php artisan make:policy IdeaPolicy --model=Idea`	Authorization policy
`php artisan make:factory IdeaFactory`	Model factory for testing
`php artisan make:seeder IdeaSeeder`	Database seeder
`php artisan route:list`	All registered routes with middleware
`php artisan storage:link`	Symlink storage/app/public → public/storage
`php artisan queue:work`	Process queued jobs
`php artisan db:seed`	Run database seeders
`php artisan config:clear`	Clear cached config

THE REQUEST LIFECYCLE

Every HTTP request flows through the same pipeline — knowing this helps you debug anything:

```
Browser
  ↓
public/index.php              # Single entry point
  ↓
bootstrap/app.php             # Create application instance
  ↓
HTTP Kernel                   # Global middleware (trim strings, CSRF, etc)
  ↓
Router                        # Match URI + HTTP verb to a route
  ↓
Route Middleware              # Auth, throttle, verified, etc
  ↓
Controller / Closure          # Your code runs here
  ↓
  (validates request)
  (queries database via Eloquent)
  (returns view or redirect)
  ↓
Response                      # Sent back through middleware (in reverse)
  ↓
Browser
```

BLADE SYNTAX REFERENCE

Directive	Use
`{{ $var }}`	Escaped output (safe — prevents XSS)
`{!! $var !!}`	Raw output (dangerous — only for trusted HTML)
`{{-- comment --}}`	Blade comment (not rendered to HTML)
`@if / @elseif / @else / @endif`	Conditionals
`@unless / @endunless`	Inverse of @if
`@foreach / @endforeach`	Loop over collection
`@forelse / @empty / @endforelse`	Loop with empty fallback
`@auth / @endauth`	Show block only to logged-in users
`@guest / @endguest`	Show block only to guests
`@can('update', $idea)`	Authorization check
`@csrf`	CSRF token hidden input (required on POST forms)
`@method('PUT')`	Spoof HTTP verb for non-GET/POST forms
`@error('email') ... @enderror`	Show field-specific validation error
`@extends('layouts.app')`	Inherit from a layout
`@section('content')`	Define a section
`@yield('content')`	Output a section in a layout
`@include('partials.nav')`	Include a partial template
`<x-card>...</x-card>`	Render a Blade component
`@vite(['resources/css/app.css'])`	Inject Vite-bundled assets

HTTP VERBS TO REST ACTIONS

Verb	URI	Action	Controller Method
GET	/ideas	List all	index()
GET	/ideas/create	Show form	create()
POST	/ideas	Store new	store()
GET	/ideas/{id}	Show one	show()
GET	/ideas/{id}/edit	Show edit form	edit()
PUT/PATCH	/ideas/{id}	Update	update()
DELETE	/ideas/{id}	Delete	destroy()

SECTION 1: THE FUNDAMENTALS

The first 13 episodes build your mental model of how a Laravel request flows from URL to database to screen. By the end of this section you'll have a working CRUD application with forms, validation, and controllers.

EPISODE 1: WELCOME ABOARD

What You'll Learn: The course roadmap. Jeffrey introduces the pace of the course, what you'll build (the 'Ideas' app — a lightweight roadmap voting tool), and the philosophy: build real things, learn the framework by using it, not by memorizing.

KEY CONCEPTS

- You'll finish with a deployed Laravel app on DigitalOcean via Forge.
- Section 1 covers fundamentals. Section 2 adds auth. Section 3 goes deeper. Section 4 is the final project.
- Expected pace: ~1 hour per episode once you're building the final project.
- You do NOT need to finish the course before building your own apps — start tinkering by Episode 13.

MENTAL MODEL

Laravel is opinionated in the same way a well-run kitchen is opinionated — every tool has a place and the workflow guides you. Your job early on is to learn the layout, not question it.

EPISODE 2: SET UP YOUR DEVELOPMENT ENVIRONMENT

What You'll Learn: Install the Laravel toolchain on Mac. You'll install Herd (PHP + nginx + DB bundle), the Laravel installer, Composer, and Node. Then you'll scaffold a new project and see it serve automatically at my-app.test.

KEY CONCEPTS

- **Laravel Herd** — a single native Mac/Windows app that bundles PHP, nginx, MySQL/Postgres/Redis. Auto-serves any folder in ~/Herd as `folder-name.test`.
- PHP 8.4 is the minimum for Laravel 12 (constructor property promotion, readonly, enums, property hooks).
- **Composer** — PHP's package manager. Like npm but for PHP. Comes bundled with Herd.
- **Node/NVM** — needed for Vite (asset bundling). NVM lets you swap Node versions per project.
- `laravel new` — the official Laravel installer. Prompts you for starter kit, testing framework, and database.
- SQLite is the default — zero config, stored as a file at database/database.sqlite.
- **Herd's auto .test domains** — drop a Laravel project in ~/Herd and visit project-name.test. No config.

PRACTICAL CODE

Create a new Laravel project

```
cd ~/Herd
laravel new ideas
# Prompts:
#   Starter kit?  → None (learning from scratch)
#   Testing?      → Pest
#   Database?     → SQLite
#   Run npm install? → Yes

cd ideas
npm run dev        # start Vite in one terminal
```

Check versions (run before starting)

```
php --version          # Should be 8.4+
composer --version     # Should be 2.x
node --version         # 20+ recommended
nvm use 22             # Swap Node version if needed
```

GOTCHAS & PRO TIPS

- If laravel new prompts to update, say yes — newer installers fix subtle bugs.
- If the .test domain doesn't resolve, open Herd and make sure the parked directory is ~/Herd (or wherever you put the project).
- On Windows, Herd also works — same instructions. On Linux, use Laravel Sail (Docker) instead.
- Don't install PHP globally via Homebrew if you're using Herd — it'll conflict with Herd's PHP.

MENTAL MODEL

Herd exists so you never touch a vhost config, homebrew PHP version, or MAMP panel again. Drop a folder in ~/Herd, it's served. That's the whole model.

EPISODE 3: ROUTING 101

What You'll Learn: Every Laravel request starts at routes/web.php. You'll define your first routes — closures that return strings, then views. You'll learn the difference between a URI and a URL, and why 404s are just missing route definitions.

KEY CONCEPTS

- `Route::get('/uri', function() { ... })` — defines a GET route with a closure handler.
- Routes map HTTP verb + URI to code. No match = 404.
- `view('welcome')` — renders resources/views/welcome.blade.php.
- Return types: strings are sent as text/html, arrays become JSON automatically, views render Blade templates.

PRACTICAL CODE

```
// routes/web.php
<?php
```

```php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/ideas', function () {
    return view('ideas.index');
});

Route::get('/ideas/{id}', function ($id) {
    return "Showing idea #{$id}";
});

// JSON response — array auto-converts
Route::get('/api/ping', function () {
    return ['status' => 'ok', 'time' => now()];
});
```

Named routes — reference by name instead of URL

```php
// routes/web.php
Route::get('/ideas', function () {
    return view('ideas.index');
})->name('ideas.index');

// In Blade: <a href="{{ route('ideas.index') }}">All Ideas</a>
```

GOTCHAS & PRO TIPS

- Always name your routes early — refactoring URLs later is painless if you do.
- routes/web.php has session + CSRF middleware by default. routes/api.php is stateless.
- A 404 just means 'no route matched' — check spelling and HTTP verb.
- The {id} in the URI becomes a parameter passed to your closure.

MENTAL MODEL

Routes are the switchboard of your app — every request first asks 'is there a match here?' If yes, run the code. If no, 404. Everything else is details.

EPISODE 4: LAYOUT FILES

What You'll Learn: Stop repeating your header, footer, and navigation on every page. Blade layouts let one master template define the shell, and child templates fill in the content sections.

KEY CONCEPTS

- `@extends('layouts.app')` — child inherits from a parent layout.
- `@section('content') ... @endsection` — child defines a named chunk.
- `@yield('content')` — parent outputs the named chunk.
- Blade components (<x-layout>) are the modern alternative — covered later.
- Layouts live in resources/views/layouts/ by convention.

PRACTICAL CODE

```blade
// resources/views/layouts/app.blade.php
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>@yield('title', 'Ideas')</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="bg-gray-100">
    <nav class="bg-white shadow p-4">
        <a href="{{ route('ideas.index') }}">Ideas</a>
    </nav>

    <main class="container mx-auto p-6">
        @yield('content')
    </main>
</body>
</html>
```

```blade
// resources/views/ideas/index.blade.php
@extends('layouts.app')

@section('title', 'All Ideas')

@section('content')
    <h1 class="text-3xl font-bold">All Ideas</h1>
    <p>Here's what everyone's been submitting.</p>
@endsection
```

GOTCHAS & PRO TIPS

- @yield('title', 'Default') — the second arg is a fallback if the child doesn't define the section.
- Only ONE @extends per file, must be the first line.
- Components (<x-layout>) are more composable than @extends — Jeffrey will migrate to them later.

MENTAL MODEL

Think of @extends as inheritance for templates. The parent is the shell; the child fills the gaps. Same DRY principle as class inheritance.

EPISODE 5: PASS DATA TO VIEWS

What You'll Learn: Three ways to pass PHP data into a Blade template: the second argument to view(), compact(), and chained with(). All do the same thing — pick the one that reads best for the situation.

KEY CONCEPTS

- `view('name', ['key' => $value])` — array as second arg.
- `compact('var1', 'var2')` — builds the array from variable names.
- `view('name')->with('key', $value)` — chainable, one key at a time.
- In Blade, access with {{ $key }}.

PRACTICAL CODE

```php
// routes/web.php
Route::get('/ideas', function () {
    $ideas = [
        ['title' => 'Dark mode', 'status' => 'pending'],
        ['title' => 'CSV export', 'status' => 'completed'],
    ];

    // Option 1: array
    return view('ideas.index', ['ideas' => $ideas]);

    // Option 2: compact (most common)
    return view('ideas.index', compact('ideas'));

    // Option 3: with
    return view('ideas.index')->with('ideas', $ideas);
});
```

```blade
// resources/views/ideas/index.blade.php
@extends('layouts.app')

@section('content')
    <h1>All Ideas</h1>
    <ul>
        @foreach ($ideas as $idea)
            <li>{{ $idea['title'] }} — {{ $idea['status'] }}</li>
        @endforeach
    </ul>
@endsection
```

GOTCHAS & PRO TIPS

-
- {{ }} auto-escapes HTML entities, preventing XSS. Use {!! !!} only for trusted HTML.
- You can pass any type — strings, arrays, collections, Eloquent models, even closures.

MENTAL MODEL

A view is just a function: data in, HTML out. Everything you pass via view() becomes a local variable in the Blade template.

EPISODE 6: BLADE DIRECTIVES

What You'll Learn: Blade's @-directives are compiled shortcuts for common PHP patterns. Once you know the core set — @if, @foreach, @forelse, @auth, @csrf — you rarely need raw PHP in a template.

KEY CONCEPTS

- @if / @elseif / @else / @endif — conditional rendering
- @foreach / @endforeach — loop over array or collection
- @forelse / @empty / @endforelse — loop with 'if empty' fallback in one block
- @auth / @endauth — only show to logged-in users
- @guest / @endguest — only show to visitors
- @csrf — outputs a hidden CSRF token input (required on POST forms)

- @method('PUT') — spoof HTTP verbs beyond GET/POST (HTML forms only natively support those two)

PRACTICAL CODE

```
// resources/views/ideas/index.blade.php
@forelse ($ideas as $idea)
    <div class="card">
        <h3>{{ $idea->title }}</h3>
        @if ($idea->status === 'completed')
            <span class="badge-green">Done</span>
        @elseif ($idea->status === 'in-progress')
            <span class="badge-blue">In Progress</span>
        @else
            <span class="badge-gray">Pending</span>
        @endif
    </div>
@empty
    <p>No ideas yet. Be the first!</p>
@endforelse
```

@auth and @guest

```
@auth
    <p>Welcome back, {{ auth()->user()->name }}</p>
@endauth

@guest
    <a href="{{ route('login') }}">Sign in to submit an idea</a>
@endguest
```

GOTCHAS & PRO TIPS

- @forelse is ONE of the biggest wins in Blade — replaces if(count($x)) { foreach } else { }
- @csrf is MANDATORY on POST forms — without it, Laravel returns 419 Page Expired.
- @method is needed for PUT, PATCH, DELETE — HTML forms only support GET and POST.
- Blade directives compile to plain PHP — they're not magic, just shorthand.

MENTAL MODEL

Directives are sugar for patterns you'd otherwise write dozens of times. They don't give you new powers — they remove friction from old ones.

EPISODE 7: FORMS

What You'll Learn: Build an HTML form that submits data to Laravel. You'll learn CSRF protection, method spoofing, and flash messaging so 'Idea created!' appears after a redirect.

KEY CONCEPTS

- Every POST/PUT/PATCH/DELETE form needs @csrf.
- For PUT/PATCH/DELETE, also need @method('PUT') or similar.
- `$request->input('field')` or `$request->field` to read submitted data.

- `session()->flash('success', 'Idea saved!')` — set a one-request session flag.
- `redirect()->back()->with('success', '...')` — shortcut: redirect + flash in one.

PRACTICAL CODE

```
// resources/views/ideas/create.blade.php
<form method="POST" action="{{ route('ideas.store') }}">
    @csrf

    <label>Title</label>
    <input type="text" name="title" value="{{ old('title') }}">

    <label>Description</label>
    <textarea name="description">{{ old('description') }}</textarea>

    <button type="submit">Submit Idea</button>
</form>

@if (session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
@endif
```

```
// routes/web.php
Route::post('/ideas', function (Request $request) {
    // (validation comes in Episode 11)
    $idea = Idea::create([
        'title'       => $request->input('title'),
        'description' => $request->input('description'),
    ]);

    return redirect()->route('ideas.index')
        ->with('success', 'Idea submitted!');
})->name('ideas.store');
```

Deleting — method spoofing

```
<form method="POST" action="{{ route('ideas.destroy', $idea) }}">
    @csrf
    @method('DELETE')
    <button type="submit">Delete</button>
</form>
```

GOTCHAS & PRO TIPS

- old('title') — re-populates the field after a failed validation redirect. Essential UX.
- Flash data lives for exactly ONE request after being set, then disappears.
- Never trust $request data — always validate (Episode 11).
- CSRF failure returns 419. If you see that, you forgot @csrf.

MENTAL MODEL

A form is an RPC call disguised as HTML. The browser's job is to serialize the inputs; Laravel's job is to validate, act, and redirect with a message.

EPISODE 8: DATABASES, MIGRATIONS, AND ELOQUENT

What You'll Learn: This is the core of Laravel. Migrations are version-controlled schema changes. Eloquent is the ORM — every table has a model class, and you query it with methods that read like English.

KEY CONCEPTS

- **Migrations** — PHP files that describe schema changes. up() applies, down() reverses.
- **Eloquent models** — one class per table. App\Models\Idea maps to ideas table.
- **Active Record pattern** — the model IS the row. $idea->save() writes to DB.
- `Idea::all()`, `Idea::find(1)`, `Idea::where(...)->get()` — the bread and butter of queries.
- `$guarded` — array of attributes that can't be mass-assigned. Set to [] to allow all (many pros do this).
- `when()` — conditional query clauses without ugly if/else around your builder.

PRACTICAL CODE

Create the migration

```
php artisan make:migration create_ideas_table
```

```
// database/migrations/xxxx_create_ideas_table.php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('ideas', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->string('title');
            $table->text('description');
            $table->string('status')->default('pending');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('ideas');
    }
};
```

Run migrations

```
php artisan migrate          # apply pending
php artisan migrate:rollback # undo last batch
php artisan migrate:refresh  # drop all + re-run
php artisan migrate:fresh    # drop all + re-run (faster, no rollbacks)
```

```
// app/Models/Idea.php
<?php

namespace App\Models;
```

```php
use Illuminate\Database\Eloquent\Model;

class Idea extends Model
{
    protected $guarded = [];   // allow all mass-assignment (pros do this)
    // OR
    protected $fillable = ['title', 'description', 'status', 'user_id'];
}
```

Eloquent vs query builder — same result, different expressiveness

```php
// Eloquent (preferred)
$ideas = Idea::where('status', 'pending')
    ->latest()
    ->get();

// Query Builder (equivalent)
$ideas = DB::table('ideas')
    ->where('status', 'pending')
    ->orderBy('created_at', 'desc')
    ->get();

// Create — Eloquent
$idea = Idea::create([
    'title' => 'Dark mode',
    'description' => 'Please.',
]);

// Query Builder — returns ID, not a model
$id = DB::table('ideas')->insertGetId([...]);
```

when() — conditional queries without ugly branching

```php
// BAD — breaks the fluent chain
$query = Idea::query();
if ($request->has('status')) {
    $query = $query->where('status', $request->status);
}
$ideas = $query->get();

// GOOD — when() runs the closure only if the first arg is truthy
$ideas = Idea::query()
    ->when($request->status, fn($q, $status) => $q->where('status', $status))
    ->when($request->search, fn($q, $s) => $q->where('title', 'like', "%{$s}%"))
    ->latest()
    ->get();
```

GOTCHAS & PRO TIPS

- Most prominent Laravel devs set $guarded = [] and validate explicitly in Form Requests instead of relying on $fillable — it's more explicit.
- foreignId()->constrained() auto-infers the referenced table (user_id → users).
- cascadeOnDelete() — when a user is deleted, their ideas go too.
- Always define a down() method even if you never rollback — it's documentation of the schema change.
- Eloquent returns Collections (not arrays) — get .first(), .pluck(), .map() helpers for free.

MENTAL MODEL

Migrations are git for your schema. Eloquent is a thin, expressive layer over SQL — every model method maps to a query you could write by hand, but reads better.

EPISODE 9: HTTP REQUESTS AND REST

What You'll Learn: REST is a convention for mapping the 7 common actions on a resource (list, create, store, show, edit, update, destroy) to HTTP verbs and URIs. Follow the convention and Laravel gives you controllers, routing, and tooling for free.

KEY CONCEPTS

- The 7 RESTful actions: index, create, store, show, edit, update, destroy
- index = list all, create = show form, store = save new
- show = view one, edit = show edit form, update = save changes, destroy = delete
- `Route::resource('ideas', IdeaController::class)` — auto-registers all 7.

PRACTICAL CODE

All 7 routes by hand (verbose)

```
// routes/web.php
Route::get('/ideas',            [IdeaController::class, 'index'])->name('ideas.index');
Route::get('/ideas/create',     [IdeaController::class, 'create'])->name('ideas.create');
Route::post('/ideas',           [IdeaController::class, 'store'])->name('ideas.store');
Route::get('/ideas/{idea}',     [IdeaController::class, 'show'])->name('ideas.show');
Route::get('/ideas/{idea}/edit',[IdeaController::class, 'edit'])->name('ideas.edit');
Route::put('/ideas/{idea}',     [IdeaController::class, 'update'])->name('ideas.update');
Route::delete('/ideas/{idea}',  [IdeaController::class, 'destroy'])->name('ideas.destroy');
```

Same thing, one line

```
// routes/web.php
Route::resource('ideas', IdeaController::class);
```

Inspect what was registered

```
php artisan route:list --path=ideas
```

GOTCHAS & PRO TIPS

- Only use Route::resource() when you actually need all 7 methods. If you only need show + store, write them by hand.
- Route::resource('ideas', ...)->only(['index', 'show']) — subset shortcut.
- URL parameter {idea} enables route model binding (next episode) — Laravel auto-loads the Idea from the ID.

MENTAL MODEL

REST is less about 'correct API design' and more about 'the framework will do more for you if you follow this convention.' Deviate only when you need to.

EPISODE 10: CONTROLLERS

What You'll Learn: Move your closures out of routes/web.php and into controller classes. This scales — a 20-route app in closures is unreadable; 20 routes grouped across 3 controllers is clean.

KEY CONCEPTS

- `php artisan make:controller IdeaController --resource` — scaffolds a controller with all 7 REST methods.
- `php artisan make:controller --invokable` — single-action controller with one __invoke() method.
- Inject Request into any controller method — Laravel auto-resolves it.
- Route model binding: type-hint the model (Idea $idea), Laravel auto-fetches by ID.

PRACTICAL CODE

Scaffold a resource controller

```
php artisan make:controller IdeaController --resource --model=Idea
```

```
// app/Http/Controllers/IdeaController.php
<?php

namespace App\Http\Controllers;

use App\Models\Idea;
use Illuminate\Http\Request;

class IdeaController extends Controller
{
    public function index()
    {
        return view('ideas.index', [
            'ideas' => Idea::latest()->get(),
        ]);
    }

    public function create()
    {
        return view('ideas.create');
    }

    public function store(Request $request)
    {
        $idea = Idea::create([
            'user_id'     => auth()->id(),
            'title'       => $request->input('title'),
            'description' => $request->input('description'),
        ]);

        return redirect()->route('ideas.show', $idea);
    }

    public function show(Idea $idea)        // route model binding
    {
        return view('ideas.show', compact('idea'));
    }

    public function edit(Idea $idea)
    {
        return view('ideas.edit', compact('idea'));
    }

    public function update(Request $request, Idea $idea)
    {
```

```
        $idea->update($request->only(['title', 'description']));
        return redirect()->route('ideas.show', $idea);
    }

    public function destroy(Idea $idea)
    {
        $idea->delete();
        return redirect()->route('ideas.index');
    }
}
```

Single-action controller — when ONE method is all you need

```
// php artisan make:controller DashboardController --invokable
class DashboardController extends Controller
{
    public function __invoke()
    {
        return view('dashboard', [
            'ideas' => auth()->user()->ideas,
        ]);
    }
}

// routes/web.php
Route::get('/dashboard', DashboardController::class);
```

GOTCHAS & PRO TIPS

- Route model binding ONLY works when the route parameter name matches the variable name: {idea} → $idea.
- If the model isn't found, Laravel auto-returns 404 — you don't need to check.
- Single-action controllers are great for 'view a dashboard' or 'export a report' actions that don't fit REST.
- Don't put business logic in controllers — delegate to action classes (Episode 37).

MENTAL MODEL

A controller is a dispatcher, not a worker. Its job is to accept a request, delegate to models/actions, and return a response. Thin controllers age well.

EPISODE 11: REQUEST VALIDATION

What You'll Learn: Never trust user input. $request->validate() runs rules against submitted data; if anything fails, Laravel redirects back with errors and old input automatically.

KEY CONCEPTS

- `$request->validate(['title' => 'required|min:5'])` — validates and returns only validated fields.
- On failure: redirect back, flash errors, flash old input — Blade picks it up via old() and @error.
- Rules can be strings ('required|email') or arrays (['required', 'email']).
- @error('field') — shows the first error for a field.

PRACTICAL CODE

```php
// app/Http/Controllers/IdeaController.php
public function store(Request $request)
{
    $validated = $request->validate([
        'title'       => ['required', 'string', 'min:5', 'max:120'],
        'description' => ['required', 'string', 'min:20'],
        'status'      => ['required', 'in:pending,in-progress,completed'],
    ]);

    $validated['user_id'] = auth()->id();
    $idea = Idea::create($validated);

    return redirect()->route('ideas.show', $idea)
        ->with('success', 'Idea created!');
}
```

```blade
// resources/views/ideas/create.blade.php
<form method="POST" action="{{ route('ideas.store') }}">
    @csrf

    <label>Title</label>
    <input type="text" name="title" value="{{ old('title') }}"
        class="@error('title') border-red-500 @enderror">
    @error('title')
        <p class="text-red-600 text-sm">{{ $message }}</p>
    @enderror

    <label>Description</label>
    <textarea name="description">{{ old('description') }}</textarea>
    @error('description')
        <p class="text-red-600 text-sm">{{ $message }}</p>
    @enderror

    <button type="submit">Submit</button>
</form>
```

GOTCHAS & PRO TIPS

- validate() throws a ValidationException on failure — caught automatically; redirects back with errors.
- $validated only contains fields you validated — safer than $request->all() for mass assignment.
- For custom messages: $request->validate($rules, $messages) with ['field.rule' => 'msg'] array.
- Always add old() to every input — without it, a failed submission wipes the user's work.

MENTAL MODEL

Validation is a gate at the front door. Everything past validation can be trusted; everything before cannot. Keep your models clean by never letting unvalidated data reach them.

EPISODE 12: FORM REQUEST CLASSES

What You'll Learn: Move validation rules out of controllers into dedicated Form Request classes. Controller methods stay thin; rules become reusable and testable.

KEY CONCEPTS

- `php artisan make:request StoreIdeaRequest` — creates a FormRequest class.
- `authorize()` — return true/false to permit or deny. Defaults to true now (changed in Laravel 10+).
- `rules()` — returns the validation rules array.
- Type-hint the FormRequest in your controller — Laravel auto-validates before the method runs.

PRACTICAL CODE

Generate the request class

```
php artisan make:request StoreIdeaRequest
```

```
// app/Http/Requests/StoreIdeaRequest.php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreIdeaRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();   // only logged-in users may submit
    }

    public function rules(): array
    {
        return [
            'title'       => ['required', 'string', 'min:5', 'max:120'],
            'description' => ['required', 'string', 'min:20'],
            'status'      => ['required', 'in:pending,in-progress,completed'],
        ];
    }

    public function messages(): array
    {
        return [
            'title.min' => 'Give your idea a longer title — be descriptive!',
        ];
    }
}
```

Controller — before and after

```
// BEFORE — rules mixed with logic
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => ['required', 'min:5'],
        // ... 10 more rules
    ]);
    Idea::create($validated);
}

// AFTER — rules live in StoreIdeaRequest
public function store(StoreIdeaRequest $request)
{
```

```
    Idea::create($request->validated());
    return redirect()->route('ideas.index');
}
```

GOTCHAS & PRO TIPS

- Type-hinting StoreIdeaRequest in the method triggers validation automatically — you don't call validate() manually.
- authorize() returning false throws a 403 — perfect for resource ownership checks.
- Use $request->validated() to get just the validated fields, not $request->all().
- Form Requests make controllers so thin they become almost declarative. This is the Laravel way.

MENTAL MODEL

A FormRequest is a validation-scoped object. It answers two questions: 'Is this user allowed?' and 'Is this data valid?' If both pass, the controller runs.

EPISODE 13: A BRIEF DAISYUI DETOUR

What You'll Learn: Tailwind CSS alone is verbose for common UI (buttons, inputs, modals). DaisyUI is a plugin that adds pre-styled components (btn, card, input, modal) as single Tailwind classes.

KEY CONCEPTS

- **DaisyUI** — Tailwind plugin. Install via npm. Adds classes like btn, btn-primary, card, modal.
- Themes: DaisyUI ships with dark/light/cupcake/etc. Configure in tailwind.config.js.
- You still get full Tailwind — DaisyUI is additive, not a replacement.

PRACTICAL CODE

Install DaisyUI

```
npm install -D daisyui@latest
```

```
// resources/css/app.css
@import "tailwindcss";
@plugin "daisyui" {
  themes: light --default, dark;
}
```

Use DaisyUI components

```
<button class="btn btn-primary">Submit Idea</button>
<button class="btn btn-ghost">Cancel</button>

<div class="card bg-base-100 shadow-xl">
    <div class="card-body">
        <h2 class="card-title">{{ $idea->title }}</h2>
        <p>{{ $idea->description }}</p>
    </div>
</div>

<input type="text" class="input input-bordered" placeholder="Title">
```

GOTCHAS & PRO TIPS

- Tailwind 4 uses @import and @plugin syntax — don't mix with the Tailwind 3 config format.
- Run npm run dev in a separate terminal — Vite watches your files and rebuilds CSS on save.
- DaisyUI is optional — pure Tailwind is completely fine. Jeffrey uses it to keep styling out of the spotlight.

MENTAL MODEL

Tailwind is alphabet; DaisyUI is words. You can spell everything from letters, but having 'btn' pre-styled saves a hundred keystrokes per form.

SECTION 2: AUTHENTICATION AND AUTHORIZATION

Authentication = who are you? Authorization = what are you allowed to do? Laravel handles the first with sessions and hashing; the second with gates and policies. This section builds both from scratch so you understand the machinery.

EPISODE 14: AUTHENTICATION EXPLAINED

What You'll Learn: Build registration and login by hand — no starter kit. You'll hash passwords, log users in, regenerate sessions to prevent hijacking, and wire up logout. By the end, a sessions controller feels like just another resource controller.

KEY CONCEPTS

- `Hash::make($password)` — bcrypt a password. Never store plain text, ever.
- `Auth::login($user)` — log a user in manually (after registration).
- `Auth::attempt(['email' => ..., 'password' => ...])` — check credentials + log in. Returns true/false.
- `$request->session()->regenerate()` — new session ID on login (prevents session fixation).
- `Auth::logout()` + `session()->invalidate()` + `session()->regenerateToken()` — the full logout trio.
- `redirect()->intended('/dashboard')` — sends user to where they originally wanted to go after login.
- Treat the sessions controller as REST: create (show login form), store (log in), destroy (log out).

PRACTICAL CODE

```
// routes/web.php
// Registration
Route::get('/register', [RegisteredUserController::class, 'create'])->name('register');
Route::post('/register', [RegisteredUserController::class, 'store']);

// Login (sessions controller – RESTful)
Route::get('/login', [SessionsController::class, 'create'])->name('login');
Route::post('/login', [SessionsController::class, 'store']);
Route::post('/logout', [SessionsController::class, 'destroy'])->name('logout');
```

```
// app/Http/Controllers/RegisteredUserController.php
public function store(Request $request)
{
    $validated = $request->validate([
        'name'     => ['required', 'string', 'max:255'],
        'email'    => ['required', 'email', 'unique:users'],
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name'     => $validated['name'],
        'email'    => $validated['email'],
```

```
        'password' => Hash::make($validated['password']),
    ]);

    Auth::login($user);

    return redirect()->route('dashboard');
}
// app/Http/Controllers/SessionsController.php
public function store(Request $request)
{
    $credentials = $request->validate([
        'email'    => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (! Auth::attempt($credentials, $request->boolean('remember'))) {
        throw ValidationException::withMessages([
            'email' => 'Those credentials don\'t match our records.',
        ]);
    }

    $request->session()->regenerate();

    return redirect()->intended(route('ideas.index'));
}

public function destroy(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/');
}
```

GOTCHAS & PRO TIPS

- ALWAYS hash with Hash::make(). Plain passwords are a breach waiting to happen.
- ALWAYS regenerate() the session on login — prevents session fixation attacks.
- 'password.confirmed' requires a 'password_confirmation' field on the form.
- intended() falls back to '/' if no intended URL was stored (e.g. direct login visit).
- Logout needs ALL THREE: Auth::logout(), session invalidate, token regenerate. Omit and you leak state.

MENTAL MODEL

Authentication is a state machine. Registration creates a user. Login creates a session tied to that user. Logout destroys the session. Everything else is UI around those three events.

EPISODE 15: REQUIRE AUTHENTICATION WITH MIDDLEWARE

What You'll Learn: Protect routes with middleware. The auth middleware blocks unauthenticated requests and redirects them to login. Group related routes so you don't repeat yourself.

KEY CONCEPTS

- **Middleware** = a filter that runs before (or after) a route handler. Can short-circuit the request.
- `->middleware('auth')` — redirects guests to /login.
- `->middleware('guest')` — only allows guests (e.g. login page hidden from logged-in users).
- `Route::middleware('auth')->group(fn() => ...)` — apply to many routes at once.

PRACTICAL CODE

Protect individual routes

```
// routes/web.php
Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware('auth')
    ->name('dashboard');

Route::post('/ideas', [IdeaController::class, 'store'])
    ->middleware('auth');
```

Groups — cleaner when you have many protected routes

```
// routes/web.php
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
    Route::resource('ideas', IdeaController::class)->except(['index', 'show']);
});

Route::middleware('guest')->group(function () {
    Route::get('/login', [SessionsController::class, 'create'])->name('login');
    Route::get('/register', [RegisteredUserController::class, 'create'])->name('register');
});
```

Write your own middleware

```
php artisan make:middleware EnsureIdeaOwner
```

GOTCHAS & PRO TIPS

- The 'auth' middleware redirects guests to the login route named 'login' — if you rename, update config.
- Use ->except([...]) or ->only([...]) on resource routes to customize which are protected.
- Middleware runs in a specific order — global → group → route. Check bootstrap/app.php.
- Custom middleware is registered in bootstrap/app.php via $middleware->alias([...]) in Laravel 11+.

MENTAL MODEL

Middleware is a pipe. Request goes in, passes through filters (auth, csrf, throttle), reaches the controller, and the response flows back through the same pipe. If a filter short-circuits, the controller never runs.

EPISODE 16: ELOQUENT RELATIONSHIPS

What You'll Learn: Model relationships let you express 'a user has many ideas' or 'an idea belongs to a user' in code. Laravel generates the SQL joins for you.

KEY CONCEPTS

- **hasMany** — one User has many Ideas.
- **belongsTo** — inverse: each Idea belongs to one User.
- **hasOne** — one User has one Profile.
- **belongsToMany** — many-to-many via pivot table (User ↔ Role).
- `foreignId('user_id')->constrained()` — foreign key in migration, auto-references users.id.
- `->with('user')` — eager loading. Avoids N+1 queries.

PRACTICAL CODE

```php
// app/Models/User.php
class User extends Authenticatable
{
    public function ideas(): HasMany
    {
        return $this->hasMany(Idea::class);
    }
}

// app/Models/Idea.php
class Idea extends Model
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Use the relationships

```php
$user = User::find(1);
$user->ideas;              // Collection of Idea models — lazy loaded on access

$idea = Idea::find(1);
$idea->user->name;         // "Benjamin"

// Create via relationship — auto-sets user_id
$user->ideas()->create([
    'title'       => 'Dark mode',
    'description' => 'Please.',
]);
```

The N+1 problem — AVOID

```php
// ❌ BAD — 1 query for ideas + N queries (one per idea) for users
$ideas = Idea::all();
foreach ($ideas as $idea) {
    echo $idea->user->name;  // triggers a query EACH loop
}

// ✅ GOOD — 2 queries total regardless of result count
$ideas = Idea::with('user')->get();
foreach ($ideas as $idea) {
    echo $idea->user->name;  // already loaded
}
```

GOTCHAS & PRO TIPS

- N+1 queries are the #1 Laravel performance bug. Always eager load with with() when you'll use the relationship.
- foreignId('user_id')->constrained() auto-infers — but you can override: ->constrained('admins').
- $user->ideas is a Collection. $user->ideas() returns a query builder — chain ->where(), ->get() on it.
- Install Laravel Debugbar or Telescope to see every query your page runs — instantly spots N+1.

MENTAL MODEL

Relationships are how you model the real world in your database. 'A user has many ideas' isn't database jargon — it's English. Eloquent just gives you methods that mirror those sentences.

EPISODE 17: AUTHORIZATION USING GATES

What You'll Learn: Gates are closures that answer yes/no questions: 'can this user edit this idea?' They live in a service provider and you call them with Gate::allows() or the @can directive.

KEY CONCEPTS

- `Gate::define('update-idea', fn($user, $idea) => $user->id === $idea->user_id)` — define a gate.
- `Gate::allows('update-idea', $idea)` — check it in a controller.
- `@can('update-idea', $idea) ... @endcan` — Blade directive.
- `$this->authorize('update-idea', $idea)` — in controllers; aborts 403 if denied.

PRACTICAL CODE

```php
// app/Providers/AppServiceProvider.php
public function boot(): void
{
    Gate::define('update-idea', function (User $user, Idea $idea) {
        return $user->id === $idea->user_id;
    });

    Gate::define('delete-idea', function (User $user, Idea $idea) {
        return $user->id === $idea->user_id;
    });
}
```

Use in controller

```php
public function update(UpdateIdeaRequest $request, Idea $idea)
{
    $this->authorize('update-idea', $idea);  // throws 403 if not allowed

    $idea->update($request->validated());
    return redirect()->route('ideas.show', $idea);
}
```

Use in Blade

```
@can('update-idea', $idea)
```

```
    <a href="{{ route('ideas.edit', $idea) }}">Edit</a>
@endcan

@cannot('update-idea', $idea)
    <p class="text-gray-500">You can't edit this.</p>
@endcannot
```

GOTCHAS & PRO TIPS

- Gates are good for one-off or non-model checks (e.g. 'is-admin'). For model-tied rules, use Policies (next episode).
- Authorization fails = 403 response. Laravel renders errors/403.blade.php if present.
- Gates receive the authenticated User automatically as the first argument.

MENTAL MODEL

A gate is a named permission check — it's a lookup table of rules. When you ask 'can this user do X?', Laravel runs the closure with that name and returns yes or no.

EPISODE 18: AUTHORIZATION USING POLICIES

What You'll Learn: Policies are classes that group all authorization methods for a specific model. Scale better than gates when you have lots of rules around a resource.

KEY CONCEPTS

- `php artisan make:policy IdeaPolicy --model=Idea` — scaffolds a class with view, create, update, delete methods.
- Laravel auto-resolves the policy: IdeaController::update() + Idea model → IdeaPolicy::update().
- `$this->authorize('update', $idea)` — in controller; uses the policy.
- `@can('update', $idea)` — same in Blade, now using the policy.
- `Route::can('update', 'idea')` — apply policy check as route middleware.

PRACTICAL CODE

Generate the policy

```
php artisan make:policy IdeaPolicy --model=Idea
```

```
// app/Policies/IdeaPolicy.php
<?php

namespace App\Policies;

use App\Models\Idea;
use App\Models\User;

class IdeaPolicy
{
    public function viewAny(User $user): bool
    {
        return true;  // anyone can see the list
    }

    public function view(User $user, Idea $idea): bool
    {
```

```
        return true;  // anyone can view a single idea
    }

    public function create(User $user): bool
    {
        return true;  // any logged-in user can submit
    }

    public function update(User $user, Idea $idea): bool
    {
        return $user->id === $idea->user_id;
    }

    public function delete(User $user, Idea $idea): bool
    {
        return $user->id === $idea->user_id || $user->is_admin;
    }
}
```

Controller — no registration needed, Laravel auto-resolves

```
public function edit(Idea $idea)
{
    $this->authorize('update', $idea);
    return view('ideas.edit', compact('idea'));
}

public function update(UpdateIdeaRequest $request, Idea $idea)
{
    $this->authorize('update', $idea);
    $idea->update($request->validated());
    return redirect()->route('ideas.show', $idea);
}
```

Route-level authorization

```
// routes/web.php
Route::put('/ideas/{idea}', [IdeaController::class, 'update'])
    ->can('update', 'idea')
    ->name('ideas.update');
```

GOTCHAS & PRO TIPS

- Policy method names should match REST actions: view, create, update, delete. Laravel auto-maps them.
- Gates run first; if a gate denies, the policy never runs. They coexist.
- Add a before() method on the policy to grant super-admins access to everything: if ($user->is_admin) return true;
- In Laravel 11+, policies are auto-discovered by naming convention — no manual registration needed.

MENTAL MODEL

A gate is a standalone rule; a policy is a cluster of rules attached to a model. If the question is 'can this user do X to this Idea?', the answer lives in IdeaPolicy.

SECTION 3: DIGGING DEEPER

Beyond CRUD — the infrastructure pieces that turn a tutorial app into a real one: asset bundling, notifications, background jobs, and tests.

EPISODE 19: FRONTEND ASSET BUNDLING WITH VITE

What You'll Learn: Vite bundles CSS and JS for production and provides hot module reload in development. Laravel includes it pre-configured — you just point @vite at your entry files.

KEY CONCEPTS

- **Vite** — modern bundler (replaces Webpack/Mix). Fast dev server, optimized prod builds.
- `@vite(['resources/css/app.css', 'resources/js/app.js'])` — injects the right tags.
- `npm run dev` — starts Vite dev server with HMR.
- `npm run build` — production build, outputs to public/build/.
- **HMR** — Hot Module Reload. Save a Blade/CSS/JS file, browser updates without full reload.

PRACTICAL CODE

```
// resources/views/layouts/app.blade.php
<!DOCTYPE html>
<html>
<head>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body>
    @yield('content')
</body>
</html>
```

```
// resources/js/app.js
import './bootstrap';
import Alpine from 'alpinejs';

window.Alpine = Alpine;
Alpine.start();
```

```
// resources/css/app.css
@import "tailwindcss";
@plugin "daisyui";
```

Two terminals — keep both running during development

```
# Terminal 1 — Laravel (Herd handles this automatically)
# Terminal 2 — Vite
npm run dev

# For production deploy
npm run build
```

GOTCHAS & PRO TIPS

- If your CSS isn't updating, check Vite is running and @vite() is in the layout.

- Production needs npm run build — without it, @vite looks for a dev server that isn't there.
- HMR fails silently if the Vite terminal isn't running — nothing breaks, just stops updating.
- Vite config lives in vite.config.js — add new entries to input: [...] to bundle more files.

MENTAL MODEL

Vite is just a fancy file watcher + bundler. In dev it serves files fast; in prod it concatenates and minifies. @vite() is the glue that tells Blade which files to inject.

EPISODE 20: NOTIFICATIONS

What You'll Learn: Send emails, Slack messages, database entries, or broadcasts with a single line of code. Laravel's notification system abstracts 'tell the user something happened' across channels.

KEY CONCEPTS

- `php artisan make:notification IdeaStatusChanged` — creates a notification class.
- Channels: mail, database, broadcast, slack, vonage (SMS), or custom.
- `$user->notify(new IdeaStatusChanged($idea))` — dispatches via the channels you define.
- `Notification::send($users, new AnnouncementNotification())` — for collections.

PRACTICAL CODE

Generate the notification

```
php artisan make:notification IdeaStatusChanged
```

```
// app/Notifications/IdeaStatusChanged.php
<?php

namespace App\Notifications;

use App\Models\Idea;
use Illuminate\Notifications\Notification;
use Illuminate\Notifications\Messages\MailMessage;

class IdeaStatusChanged extends Notification
{
    public function __construct(public Idea $idea) {}

    public function via($notifiable): array
    {
        return ['mail', 'database'];
    }

    public function toMail($notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject("Your idea '{$this->idea->title}' was updated")
            ->line("Status changed to: {$this->idea->status}")
            ->action('View idea', route('ideas.show', $this->idea));
    }
```

```
    public function toArray($notifiable): array
    {
        return [
            'idea_id' => $this->idea->id,
            'status'  => $this->idea->status,
        ];
    }
}
```

Send it

```
// In a controller or action
$idea->user->notify(new IdeaStatusChanged($idea));

// To many users
Notification::send(User::admins()->get(), new AnnouncementNotification());
```

GOTCHAS & PRO TIPS

- Database channel requires a migration: php artisan notifications:table + migrate.
- Mail requires MAIL_* env vars set — use Mailtrap for local dev.
- Use 'broadcast' channel + Echo for real-time in-app notifications (needs Pusher/Reverb).
- Notifications implement ShouldQueue automatically to avoid blocking the request.

MENTAL MODEL

A notification is a message wrapped in a 'how do I deliver this?' dropdown. You write the content once; Laravel handles email/DB/SMS routing.

EPISODE 21: WHEN TO QUEUE IT UP

What You'll Learn: Slow operations (sending email, hitting 3rd-party APIs, processing images) should not block the HTTP response. Queue them as background jobs and return to the user instantly.

KEY CONCEPTS

- **Jobs** — classes that implement ShouldQueue. Laravel runs them on a worker.
- `php artisan make:job ProcessIdeaImage` — scaffolds a job class.
- `ProcessIdeaImage::dispatch($idea)` — queue the job.
- `php artisan queue:work` — start a worker that processes jobs.
- Connection drivers: sync (immediate), database, redis, sqs. Set QUEUE_CONNECTION in .env.

PRACTICAL CODE

Generate a job

```
php artisan make:job ProcessIdeaImage
php artisan queue:table && php artisan migrate   # if using DB driver
```

```
// app/Jobs/ProcessIdeaImage.php
<?php

namespace App\Jobs;
```

```php
use App\Models\Idea;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;

class ProcessIdeaImage implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue;

    public function __construct(public Idea $idea) {}

    public function handle(): void
    {
        // Resize, compress, store — all the slow stuff
        $path = $this->idea->image_path;
        // ... image processing ...
    }
}
```

Dispatch from a controller

```php
public function store(StoreIdeaRequest $request)
{
    $idea = Idea::create($request->validated());

    ProcessIdeaImage::dispatch($idea);   // fire and forget
    // user gets redirect immediately, image processes in background

    return redirect()->route('ideas.show', $idea);
}
```

Run the worker

```bash
php artisan queue:work         # foreground, for dev
php artisan queue:work --daemon # long-running
# In production, use Supervisor or Forge's "daemon" feature
```

GOTCHAS & PRO TIPS

- QUEUE_CONNECTION=sync in .env runs jobs immediately (no queue) — perfect for local dev without a worker.
- Failed jobs go to a failed_jobs table (needs migration). Retry with queue:retry.
- Jobs are serialized — pass models by instance, Laravel re-fetches them. Don't pass closures.
- Laravel Horizon is the production dashboard for Redis queues — worth learning post-course.

MENTAL MODEL

A queue is a to-do list that another process works off. Your web request adds tasks; the worker crosses them off. The user never waits.

EPISODE 22: HOW TO GET STARTED TESTING YOUR CODE

What You'll Learn: Testing lets you change code without fear. Pest is Laravel's default test runner — clean, readable, and built on PHPUnit. Feature tests exercise full HTTP requests; unit tests isolate single classes.

KEY CONCEPTS

- **Pest** — PHP test framework. Write tests as it('does x', fn() => ...);
- **Feature tests** — hit routes, assert responses. Test 'the feature works'.
- **Unit tests** — test one class in isolation. Test 'this calculator adds correctly'.
- `use RefreshDatabase` — trait that wipes DB between tests.
- `$this->actingAs($user)` — simulate logged-in user.
- `Idea::factory()->create()` — factory generates fake models.

PRACTICAL CODE

Run tests

```
php artisan test
php artisan test --filter=IdeaTest
php artisan test --parallel
```

```
// tests/Feature/IdeaTest.php
<?php

use App\Models\Idea;
use App\Models\User;

it('displays the list of ideas', function () {
    Idea::factory()->count(3)->create();

    $response = $this->get('/ideas');

    $response->assertStatus(200);
    $response->assertSee('Ideas');
});

it('allows a logged-in user to submit an idea', function () {
    $user = User::factory()->create();

    $response = $this->actingAs($user)->post('/ideas', [
        'title'       => 'Dark mode',
        'description' => 'It would be nice to have a dark theme.',
        'status'      => 'pending',
    ]);

    $response->assertRedirect();
    $this->assertDatabaseHas('ideas', ['title' => 'Dark mode']);
});

it('blocks a guest from submitting an idea', function () {
    $response = $this->post('/ideas', ['title' => 'Hello']);
    $response->assertRedirect('/login');
});
```

```
// database/factories/IdeaFactory.php
<?php

namespace Database\Factories;

use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class IdeaFactory extends Factory
{
```

```
    public function definition(): array
    {
        return [
            'user_id'     => User::factory(),
            'title'       => fake()->sentence(6),
            'description' => fake()->paragraph(),
            'status'      => fake()->randomElement(['pending', 'in-progress',
'completed']),
        ];
    }
}
```

GOTCHAS & PRO TIPS

- Add RefreshDatabase to the Pest.php file once; every test gets a fresh DB for free.
- Use SQLite in-memory for tests (DB_CONNECTION=sqlite, DB_DATABASE=:memory:) — 10x faster.
- Factories are the bedrock — define them once per model and your tests become painless.
- Name tests as sentences: 'it displays the list of ideas' — they double as documentation.

MENTAL MODEL

Tests are a safety net for refactoring. The first test feels slow to write; the hundredth saves you an hour of manual clicking. Invest early.

SECTION 4: FINAL PROJECT — BUILD & DEPLOY 'IDEAS'

Twenty-one episodes building and deploying a real app. You'll write an idea-voting platform from scratch — models, policies, filters, modals, file uploads, action classes, and finally deployment to DigitalOcean via Forge.

RUNNING ARCHITECTURE OF THE IDEAS APP

As you move through this section, the app evolves. Keep this map in your head:

```
MODELS
  User          — hasMany ideas, hasMany steps
  Idea          — belongsTo user, hasMany links, hasMany steps, hasOne featuredImage
  Link          — belongsTo idea (one idea → many external links)
  Step          — belongsTo idea (actionable steps)

ENUMS
  Status        — pending | in-progress | completed

CONTROLLERS
  IdeaController (resource, 7 methods)
  ProfileController (edit/update own profile)
  DashboardController (single-action)

POLICIES
  IdeaPolicy    — view, update, delete based on ownership

ACTION CLASSES (extracted business logic)
  CreateIdeaAction
  UpdateIdeaAction

REQUESTS (validation)
  StoreIdeaRequest, UpdateIdeaRequest

VIEWS
  layouts/app.blade.php
  ideas/index, show, create, edit
  components/card, modal, form-*

ROUTES
  /                       → home (list + filter)
  /ideas/{idea}           → show
  /dashboard              → user's own ideas
  /profile                → edit profile
  + auth routes
```

EPISODE 23: FINAL PROJECT SETUP

What You'll Learn: Start the Ideas app from a fresh install. Plan the schema on paper before writing a single migration.

KEY CONCEPTS

- Fresh Laravel install with Pest and SQLite.

- Plan models first. Don't touch code until you know the schema.
- Domain: users submit ideas; ideas have a status (pending/in-progress/completed); ideas can have supporting links and actionable steps.

PRACTICAL CODE

```
cd ~/Herd
laravel new ideas
cd ideas
npm install && npm run dev

# In another terminal
php artisan serve  # or just visit ideas.test via Herd
```

Sketch the schema before coding

```
users       (id, name, email, password, timestamps)
ideas       (id, user_id, title, description, status, featured_image, timestamps)
links       (id, idea_id, url, title, timestamps)
steps       (id, idea_id, order, description, completed_at, timestamps)
```

MENTAL MODEL

Spending 15 minutes planning the schema saves 3 hours of migration refactoring later. Think in nouns and verbs: what things exist, how do they relate, what can they do?

EPISODE 24: DESIGN YOUR MODEL LAYER

What You'll Learn: Create the Idea model, the status enum, and the relationships. Get the data model right before any UI.

KEY CONCEPTS

- `php artisan make:model Idea -mfs` — model + migration + factory + seeder.
- **PHP 8.4 enum** as the status type — replaces string validation, catches typos at compile time.
- Cast the status column to the enum in the model's $casts array.

PRACTICAL CODE

```php
// app/Enums/Status.php
<?php

namespace App\Enums;

enum Status: string
{
    case Pending    = 'pending';
    case InProgress = 'in-progress';
    case Completed  = 'completed';

    public function label(): string
    {
        return match ($this) {
            self::Pending    => 'Pending',
            self::InProgress => 'In Progress',
            self::Completed  => 'Completed',
        };
    }
```

```
    public function color(): string
    {
        return match ($this) {
            self::Pending    => 'gray',
            self::InProgress => 'blue',
            self::Completed  => 'green',
        };
    }
}
```

```
// database/migrations/xxxx_create_ideas_table.php
Schema::create('ideas', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('title');
    $table->text('description');
    $table->string('status')->default('pending');
    $table->string('featured_image')->nullable();
    $table->timestamps();
});
```

```
// app/Models/Idea.php
<?php

namespace App\Models;

use App\Enums\Status;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Idea extends Model
{
    protected $guarded = [];

    protected $casts = [
        'status' => Status::class,
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function links(): HasMany
    {
        return $this->hasMany(Link::class);
    }

    public function steps(): HasMany
    {
        return $this->hasMany(Step::class)->orderBy('order');
    }
}
```

MENTAL MODEL

Enums turn 'magic strings' into types. If you write Status::Pending instead of 'pending', your IDE autocompletes and typos become compile errors.

EPISODE 25: TAILWIND THEME SETUP AND INITIAL UI

What You'll Learn: Set up the base Tailwind theme, pick a color palette, build the navbar and layout. This is the last time you touch global styles for a while.

KEY CONCEPTS

- Configure DaisyUI themes in app.css.
- Build a reusable navbar as a Blade partial or component.
- Keep a consistent spacing scale — Tailwind's defaults are fine; don't invent new ones per page.

PRACTICAL CODE

```
// resources/views/layouts/app.blade.php
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <title>{{ config('app.name') }}</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="min-h-screen bg-base-200">
    @include('partials.nav')

    <main class="container mx-auto max-w-4xl p-6">
        @yield('content')
    </main>
</body>
</html>
```

```
// resources/views/partials/nav.blade.php
<nav class="navbar bg-base-100 shadow">
    <div class="flex-1">
        <a href="{{ route('ideas.index') }}" class="btn btn-ghost text-xl">Ideas</a>
    </div>
    <div class="flex-none">
        @auth
            <a href="{{ route('dashboard') }}" class="btn btn-ghost">Dashboard</a>
            <form method="POST" action="{{ route('logout') }}" class="inline">
                @csrf
                <button class="btn btn-ghost">Log out</button>
            </form>
        @else
            <a href="{{ route('login') }}" class="btn btn-ghost">Log in</a>
            <a href="{{ route('register') }}" class="btn btn-primary">Register</a>
        @endauth
    </div>
</nav>
```

MENTAL MODEL

Design system first, features second. Nothing kills velocity like redesigning the button style in Episode 40 because you skipped it in Episode 25.

EPISODE 26: BROWSER TESTING REGISTRATION WITH PEST

What You'll Learn: Pest 4 added real browser testing — it spins up a headless Chrome, loads your actual pages, clicks buttons. Write a registration happy-path test.

KEY CONCEPTS

- **Pest 4 browser testing** — real Chrome, not a fake HTTP request. Catches JS bugs.
- `$browser->visit('/register')->type(...)->click(...)` — fluent API.
- Slower than HTTP tests — use sparingly for critical flows (auth, checkout).

PRACTICAL CODE

```
composer require pestphp/pest-plugin-browser --dev
php artisan pest:browser-install
```

```
// tests/Browser/RegistrationTest.php
<?php

use function Pest\Browser\visit;

it('lets a new user register', function () {
    visit('/register')
        ->fill('name', 'Benjamin')
        ->fill('email', 'ben@example.com')
        ->fill('password', 'secret12345')
        ->fill('password_confirmation', 'secret12345')
        ->click('Register')
        ->assertPath('/dashboard')
        ->assertSee('Welcome, Benjamin');
});
```

MENTAL MODEL

HTTP tests answer 'did the server respond right?'. Browser tests answer 'did the user get what they expected?' — which includes JS, CSS, and interactions.

EPISODE 27: FLASH MESSAGING AND INTERACTIVITY WITH ALPINEJS

What You'll Learn: Alpine is a tiny JS library for sprinkling interactivity into Blade. Use it for toasts, modals, dropdowns — tasks where a full SPA is overkill.

KEY CONCEPTS

- **Alpine directives** — x-data, x-show, x-transition, x-on:click.
- `x-data="{ open: false }"` — declares reactive state on an element.
- `x-show='open'`, `@click='open = !open'` — reactivity.
- Combine with flash messages: x-init hides the toast after 3 seconds.

PRACTICAL CODE

```
// resources/views/partials/flash.blade.php
@if (session('success'))
    <div
        x-data="{ show: true }"
        x-show="show"
        x-init="setTimeout(() => show = false, 3000)"
        x-transition
```

```
        class="alert alert-success fixed top-4 right-4 w-auto">
        {{ session('success') }}
    </div>
@endif
```

Collapsible card

```
<div x-data="{ open: false }" class="card bg-base-100">
    <button @click="open = ! open" class="p-4 font-bold">
        {{ $idea->title }}
    </button>
    <div x-show="open" x-transition class="p-4 border-t">
        {{ $idea->description }}
    </div>
</div>
```

MENTAL MODEL

Alpine is 'HTML with a memory'. You write declarative markup; Alpine makes small parts of it reactive. Perfect for the 80% of interactivity that doesn't justify Vue or React.

EPISODE 28: IDEA CARDS

What You'll Learn: Extract the idea card markup into a reusable Blade component. <x-card :idea="$idea" /> is cleaner than 40 lines of HTML repeated.

KEY CONCEPTS

- `php artisan make:component Card` — creates the component class + view.
- Anonymous components (view-only, no class) go in resources/views/components/.
- Pass data via attributes: <x-card :idea="$idea" />. Use $idea inside the component.

PRACTICAL CODE

```
php artisan make:component Card --view
```

```
// resources/views/components/card.blade.php
@props(['idea'])

<div class="card bg-base-100 shadow hover:shadow-lg transition">
    @if ($idea->featured_image)
        <figure>
            <img src="{{ asset('storage/' . $idea->featured_image) }}" alt="">
        </figure>
    @endif
    <div class="card-body">
        <h2 class="card-title">
            <a href="{{ route('ideas.show', $idea) }}">{{ $idea->title }}</a>
        </h2>
        <p>{{ Str::limit($idea->description, 120) }}</p>
        <div class="flex justify-between items-center">
            <span class="badge badge-{{ $idea->status->color() }}">
                {{ $idea->status->label() }}
            </span>
            <span class="text-sm text-gray-500">
                by {{ $idea->user->name }}
            </span>
        </div>
    </div>
</div>
```

Use the component

```
<div class="grid md:grid-cols-2 gap-4">
    @foreach ($ideas as $idea)
        <x-card :idea="$idea" />
    @endforeach
</div>
```

MENTAL MODEL

A component is a Blade function — inputs as attributes, HTML as output. If you're copying 10+ lines of markup across views, it belongs in a component.

EPISODE 29: IDEA FILTERING

What You'll Learn: Add status filter pills with live counts. Use when() for conditional queries. Extract the query into a model scope for reusability.

KEY CONCEPTS

- `Idea::filter($filters)` — extract the whole query chain into a static method.
- Group + count for status pills: Idea::select('status', DB::raw('count(*) as total'))->groupBy('status').
- Validate the ?status=foo query string against the enum to prevent SQL injection.

PRACTICAL CODE

```
// app/Models/Idea.php
public function scopeFilter($query, array $filters)
{
    $query->when($filters['status'] ?? null,
        fn($q, $status) => $q->where('status', $status));

    $query->when($filters['search'] ?? null,
        fn($q, $search) => $q->where('title', 'like', "%{$search}%"));

    return $query;
}

public static function statusCounts(): array
{
    return self::select('status', DB::raw('count(*) as total'))
        ->groupBy('status')
        ->pluck('total', 'status')
        ->toArray();
}
```

Controller

```
public function index(Request $request)
{
    $validated = $request->validate([
        'status' => ['nullable', Rule::enum(Status::class)],
        'search' => ['nullable', 'string', 'max:100'],
    ]);

    return view('ideas.index', [
        'ideas'  => Idea::with('user')->filter($validated)->latest()->paginate(12),
        'counts' => Idea::statusCounts(),
    ]);
```

```
}
```

Status pills

```
<div class="flex gap-2">
    <a href="{{ route('ideas.index') }}"
       class="badge {{ ! request('status') ? 'badge-primary' : 'badge-ghost' }}">
        All ({{ array_sum($counts) }})
    </a>
    @foreach (App\Enums\Status::cases() as $status)
        <a href="{{ route('ideas.index', ['status' => $status->value]) }}"
           class="badge {{ request('status') === $status->value ? 'badge-primary' :
'badge-ghost' }}">
            {{ $status->label() }} ({{ $counts[$status->value] ?? 0 }})
        </a>
    @endforeach
</div>
```

GOTCHAS & PRO TIPS

- Rule::enum(Status::class) is the cleanest way to validate against an enum.
- when() with a closure only runs the callback when the first arg is truthy — no ternaries needed.
- pluck('total', 'status') returns ['pending' => 7, 'in-progress' => 3, ...] — perfect for badge counts.

MENTAL MODEL

Filtering is just conditional WHERE clauses. when() lets you chain them fluently without if/else breaking the builder pattern.

EPISODE 30: SHOW A SINGLE IDEA

What You'll Learn: Route model binding auto-fetches the idea from {idea} in the URL. Build a detailed show page with the description, links, and steps.

KEY CONCEPTS

- `public function show(Idea $idea)` — Laravel sees the type hint and auto-fetches.
- To bind by slug instead of id: override getRouteKeyName() on the model to return 'slug'.

PRACTICAL CODE

```
// app/Models/Idea.php
// Optional — if you want /ideas/dark-mode instead of /ideas/42
public function getRouteKeyName(): string
{
    return 'slug';
}
```

```
// resources/views/ideas/show.blade.php
@extends('layouts.app')

@section('content')
    <article class="prose max-w-none">
        <h1>{{ $idea->title }}</h1>
        <p class="text-sm text-gray-500">
            by {{ $idea->user->name }} ·
```

```
            {{ $idea->created_at->diffForHumans() }} ·
            <span class="badge badge-{{ $idea->status->color() }}">
                {{ $idea->status->label() }}
            </span>
        </p>
        {!! nl2br(e($idea->description)) !!}

        @if ($idea->links->isNotEmpty())
            <h3>Related links</h3>
            <ul>
                @foreach ($idea->links as $link)
                    <li><a href="{{ $link->url }}">{{ $link->title }}</a></li>
                @endforeach
            </ul>
        @endif
    </article>
@endsection
```

GOTCHAS & PRO TIPS

- {!! !!} is unsafe with user input — that's why we wrap in e() first, then convert line breaks with nl2br.
- diffForHumans() is a Carbon method — outputs '3 hours ago', '2 weeks ago'.

MENTAL MODEL

Route model binding erases 5 lines of controller code per resource: the fetch, the check, the 404. Let Laravel do it.

EPISODE 31: CREATE A FUNCTIONAL MODAL WITH ALPINEJS

What You'll Learn: A modal is just a div that's hidden by default. Alpine handles the toggle; Tailwind handles the overlay.

KEY CONCEPTS

- <dialog> element is semantic; DaisyUI provides .modal and .modal-box styling.
- x-data='{ open: false }' at the top; @click='open = true' to open; @click.outside='open = false' to close.

PRACTICAL CODE

```
// resources/views/components/modal.blade.php
@props(['trigger' => 'New Idea'])

<div x-data="{ open: false }">
    <button @click="open = true" class="btn btn-primary">{{ $trigger }}</button>

    <div x-show="open" x-transition
         class="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center z-50"
         @keydown.escape.window="open = false">
        <div @click.outside="open = false"
             class="bg-base-100 p-6 rounded-lg max-w-lg w-full">
            {{ $slot }}
            <button @click="open = false" class="btn btn-ghost mt-4">Close</button>
        </div>
    </div>
```

```
</div>
```

Use the modal

```
<x-modal trigger="Submit new idea">
    <h2 class="text-2xl font-bold">New Idea</h2>
    @include('ideas.partials.form')
</x-modal>
```

MENTAL MODEL

A modal is state: is it open? Everything else — blur, transition, outside-click — decorates that binary. Alpine makes the state trivially local.

EPISODE 32: CONSTRUCT THE IDEA FORM

What You'll Learn: Build the form partial with all fields and the right names. Reuse it between create and edit.

KEY CONCEPTS

- Extract the form to resources/views/ideas/partials/form.blade.php.
- Use old() for create; $idea->field for edit. Trick: old('title', $idea->title ?? '').
- Re-use this partial from inside a modal and from a standalone edit page.

PRACTICAL CODE

```
// resources/views/ideas/partials/form.blade.php
<form method="POST"
      action="{{ isset($idea) ? route('ideas.update', $idea) : route('ideas.store') }}"
      enctype="multipart/form-data">
    @csrf
    @isset($idea) @method('PUT') @endisset

    <div class="form-control">
        <label class="label">Title</label>
        <input type="text" name="title"
            value="{{ old('title', $idea->title ?? '') }}"
            class="input input-bordered @error('title') input-error @enderror">
        @error('title') <p class="text-error text-sm">{{ $message }}</p> @enderror
    </div>

    <div class="form-control">
        <label class="label">Description</label>
        <textarea name="description" rows="5"
            class="textarea textarea-bordered @error('description') textarea-error
@enderror">{{ old('description', $idea->description ?? '') }}</textarea>
        @error('description') <p class="text-error text-sm">{{ $message }}</p> @enderror
    </div>

    <div class="form-control">
        <label class="label">Featured image</label>
        <input type="file" name="featured_image" class="file-input file-input-bordered">
    </div>

    <button type="submit" class="btn btn-primary">
        {{ isset($idea) ? 'Update' : 'Submit' }}
    </button>
</form>
```

MENTAL MODEL

DRY up forms by writing one partial and branching on whether $idea is set. Edit = create with a head start.

EPISODE 33: TEST THE CREATE IDEA FORM

What You'll Learn: Write Pest feature tests for the create flow — guest blocked, validation errors, successful submission, user gets redirected.

PRACTICAL CODE

```
// tests/Feature/CreateIdeaTest.php
<?php

use App\Models\Idea;
use App\Models\User;

it('requires authentication', function () {
    $this->post('/ideas', [])->assertRedirect('/login');
});

it('requires a title', function () {
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post('/ideas', ['description' => 'Long enough description here.'])
        ->assertSessionHasErrors('title');
});

it('saves a valid idea', function () {
    $user = User::factory()->create();

    $this->actingAs($user)->post('/ideas', [
        'title'       => 'Dark mode please',
        'description' => 'It would really help late-night users.',
        'status'      => 'pending',
    ]);

    $this->assertDatabaseHas('ideas', [
        'title'   => 'Dark mode please',
        'user_id' => $user->id,
    ]);
});
```

MENTAL MODEL

Test the UNHAPPY paths — guest access, missing fields, bad input — BEFORE the happy path. Most bugs live there.

EPISODE 34: ALLOW FOR ONE OR MANY LINKS

What You'll Learn: An idea can have multiple external links. Model it as a HasMany relationship with a dynamic form — Alpine adds/removes link rows.

KEY CONCEPTS

- Migration: links (id, idea_id, url, title, timestamps).

- Form submits an array: name='links[][url]'.
- Controller loops over $validated['links'] and creates Link records.

PRACTICAL CODE

```
// database/migrations/xxxx_create_links_table.php
Schema::create('links', function (Blueprint $table) {
    $table->id();
    $table->foreignId('idea_id')->constrained()->cascadeOnDelete();
    $table->string('url');
    $table->string('title')->nullable();
    $table->timestamps();
});
```

Dynamic form with Alpine

```
<div x-data="{ links: [{ url: '', title: '' }] }">
    <template x-for="(link, i) in links" :key="i">
        <div class="flex gap-2">
            <input :name="'links[' + i + '][url]'" x-model="link.url"
                placeholder="URL" class="input input-bordered flex-1">
            <input :name="'links[' + i + '][title]'" x-model="link.title"
                placeholder="Title" class="input input-bordered flex-1">
            <button type="button" @click="links.splice(i, 1)" class="btn
btn-ghost">×</button>
        </div>
    </template>
    <button type="button" @click="links.push({ url: '', title: '' })"
        class="btn btn-sm btn-ghost mt-2">+ Add link</button>
</div>
```

Controller handles the array

```
public function store(StoreIdeaRequest $request)
{
    $idea = auth()->user()->ideas()->create($request->safe()->except('links'));

    foreach ($request->input('links', []) as $link) {
        if ($link['url']) {
            $idea->links()->create($link);
        }
    }

    return redirect()->route('ideas.show', $idea);
}
```

MENTAL MODEL

Related data = related forms. Design the form the same way you designed the schema — parent + repeated children.

EPISODE 35: ACTIONABLE STEPS

What You'll Learn: Steps are like a sub-todo list per idea. Same pattern as links — HasMany relationship, dynamic form.

KEY CONCEPTS

- Add steps table: idea_id, order (int), description, completed_at (nullable timestamp).
- Eager load ->orderBy('order') so they display in sequence.

- 'Mark complete' button = POST to /steps/{step}/complete → update completed_at = now().

PRACTICAL CODE

```
// app/Models/Step.php
class Step extends Model
{
    protected $guarded = [];

    protected $casts = [
        'completed_at' => 'datetime',
    ];

    public function idea(): BelongsTo
    {
        return $this->belongsTo(Idea::class);
    }

    public function isCompleted(): bool
    {
        return $this->completed_at !== null;
    }
}
```

MENTAL MODEL

Steps = links with a completion state. Same mechanics, different semantics.

EPISODE 36: UPLOAD FEATURED IMAGES TO STORAGE

What You'll Learn: Laravel's Storage facade abstracts file uploads. store() picks a random filename; the 'public' disk is the default for browser-accessible files.

KEY CONCEPTS

- `$request->file('featured_image')->store('ideas', 'public')` — saves to storage/app/public/ideas/.
- `php artisan storage:link` — creates public/storage → storage/app/public symlink (runs once).
- `asset('storage/'.$path)` — generate the public URL.
- Validate: ['image', 'mimes:jpg,png,webp', 'max:2048'] — max is in kilobytes.

PRACTICAL CODE

One-time setup

```
php artisan storage:link
```

Controller

```
public function store(StoreIdeaRequest $request)
{
    $data = $request->validated();

    if ($request->hasFile('featured_image')) {
        $data['featured_image'] = $request->file('featured_image')
            ->store('ideas', 'public');
    }
```

```
    $idea = auth()->user()->ideas()->create($data);
    return redirect()->route('ideas.show', $idea);
}
```

Display

```
@if ($idea->featured_image)
    <img src="{{ asset('storage/' . $idea->featured_image) }}"
         class="w-full rounded-lg">
@endif
```

```
// app/Http/Requests/StoreIdeaRequest.php
public function rules(): array
{
    return [
        'title'          => ['required', 'string', 'max:120'],
        'description'    => ['required', 'string', 'min:20'],
        'status'         => ['required', Rule::enum(Status::class)],
        'featured_image' => ['nullable', 'image', 'max:2048'],
    ];
}
```

GOTCHAS & PRO TIPS

- Forgetting enctype='multipart/form-data' on the form means files never reach the server. Silent failure.
- Files stored via 'public' disk live at storage/app/public/ — only accessible to browsers after storage:link.
- In production, switch to S3: disk('s3'). Only the disk changes; the code stays the same.

MENTAL MODEL

Storage disks abstract 'where files live'. Local, S3, dropbox — your code doesn't care. It just asks the 'public' disk to store something.

EPISODE 37: ACTION CLASSES

What You'll Learn: When a controller method is doing too much (validate, save, upload image, send notification, dispatch job), extract the logic into an 'action class' with one public method.

KEY CONCEPTS

- Action = single-purpose invokable class. Name: verb + noun. CreateIdeaAction, PublishArticleAction.
- Makes controllers thin. Makes logic testable in isolation. Makes it reusable from jobs, commands, other controllers.
- Invokable: __invoke() method — call it like a function: ($action)($data).

PRACTICAL CODE

```
// app/Actions/CreateIdeaAction.php
<?php

namespace App\Actions;
```

```php
use App\Models\Idea;
use App\Models\User;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\DB;

class CreateIdeaAction
{
    public function __invoke(
        User $user,
        array $data,
        ?UploadedFile $image = null,
        array $links = [],
    ): Idea {
        return DB::transaction(function () use ($user, $data, $image, $links) {
            if ($image) {
                $data['featured_image'] = $image->store('ideas', 'public');
            }

            $idea = $user->ideas()->create($data);

            foreach ($links as $link) {
                if (! empty($link['url'])) {
                    $idea->links()->create($link);
                }
            }

            return $idea;
        });
    }
}
```

Controller becomes 3 lines

```php
public function store(StoreIdeaRequest $request, CreateIdeaAction $action)
{
    $idea = $action(
        user:  $request->user(),
        data:  $request->safe()->except('featured_image', 'links'),
        image: $request->file('featured_image'),
        links: $request->input('links', []),
    );

    return redirect()->route('ideas.show', $idea);
}
```

GOTCHAS & PRO TIPS

- Wrap multi-step DB writes in DB::transaction() — if any step fails, the whole thing rolls back.
- Type-hint the action in the controller — Laravel's service container auto-resolves it.
- Actions can be tested directly: (new CreateIdeaAction)($user, [...]) — no HTTP needed.
- Named arguments (PHP 8) make action calls self-documenting.

MENTAL MODEL

A controller is a dispatcher; an action is a worker. When a controller starts doing more than 'accept, delegate, respond', the extra work belongs in an action.

EPISODE 38: AUTHORIZATION IS A REQUIREMENT

What You'll Learn: Lock down update/delete/edit to the owner only. Use the policy you wrote earlier, plus @can in Blade to hide buttons users can't use.

KEY CONCEPTS

- Apply $this->authorize() at the START of every sensitive controller method — before any other work.
- Hide UI affordances with @can — don't show an Edit button the user can't use.
- Test: a different user trying to PUT /ideas/{id} should get 403.

PRACTICAL CODE

Locked-down controller methods

```
public function edit(Idea $idea)
{
    $this->authorize('update', $idea);
    return view('ideas.edit', compact('idea'));
}

public function update(UpdateIdeaRequest $request, Idea $idea, UpdateIdeaAction $action)
{
    $this->authorize('update', $idea);
    $action($idea, $request->validated());
    return redirect()->route('ideas.show', $idea);
}

public function destroy(Idea $idea)
{
    $this->authorize('delete', $idea);
    $idea->delete();
    return redirect()->route('ideas.index');
}
```

Hide UI from users who can't use it

```
@can('update', $idea)
    <a href="{{ route('ideas.edit', $idea) }}" class="btn btn-sm">Edit</a>
@endcan

@can('delete', $idea)
    <form method="POST" action="{{ route('ideas.destroy', $idea) }}" class="inline">
        @csrf @method('DELETE')
        <button class="btn btn-sm btn-error">Delete</button>
    </form>
@endcan
```

Test the denial path

```
it('forbids another user from editing an idea', function () {
    $owner = User::factory()->create();
    $other = User::factory()->create();
    $idea = Idea::factory()->for($owner)->create();

    $this->actingAs($other)
        ->get(route('ideas.edit', $idea))
        ->assertForbidden();
});
```

MENTAL MODEL

Authorization is a requirement, not a feature. Every controller method that modifies state must check permission BEFORE doing anything else.

EPISODE 39: THE EDIT IDEA MODAL

What You'll Learn: Reuse the form partial inside a modal so editing feels instant. Same Alpine + <x-modal> wrapper you built for create.

PRACTICAL CODE

```
<x-modal trigger="Edit">
    <h2 class="text-2xl font-bold mb-4">Edit Idea</h2>
    @include('ideas.partials.form', ['idea' => $idea])
</x-modal>
```

MENTAL MODEL

Good abstractions pay you back. The modal + form partial written in Episode 31–32 now get reused with zero new code.

EPISODE 40: UPDATE IDEA ACTION

What You'll Learn: Mirror CreateIdeaAction with UpdateIdeaAction — same pattern, but updates an existing model and handles replacing (not duplicating) links and images.

PRACTICAL CODE

```
// app/Actions/UpdateIdeaAction.php
<?php

namespace App\Actions;

use App\Models\Idea;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Storage;

class UpdateIdeaAction
{
    public function __invoke(
        Idea $idea,
        array $data,
        ?UploadedFile $image = null,
        array $links = [],
    ): Idea {
        return DB::transaction(function () use ($idea, $data, $image, $links) {
            if ($image) {
                if ($idea->featured_image) {
                    Storage::disk('public')->delete($idea->featured_image);
                }
                $data['featured_image'] = $image->store('ideas', 'public');
            }

            $idea->update($data);

            // Replace all links — simpler than diffing
```

```
            $idea->links()->delete();
            foreach ($links as $link) {
                if (! empty($link['url'])) {
                    $idea->links()->create($link);
                }
            }

            return $idea->fresh();
        });
    }
}
```

GOTCHAS & PRO TIPS

- Delete the old image file when replacing — otherwise storage bloats over time.
- $idea->fresh() returns a new instance with the latest DB state — safer than reading stale in-memory attributes.

MENTAL MODEL

Create and Update are siblings — write them as separate actions rather than one 'save' action with if/else. Clarity beats DRY at this scale.

EPISODE 41: EDIT YOUR PROFILE

What You'll Learn: Let users change their name, email, and password. Two forms (one for profile info, one for password) so each can validate independently.

PRACTICAL CODE

```
// app/Http/Controllers/ProfileController.php
class ProfileController extends Controller
{
    public function edit()
    {
        return view('profile.edit');
    }

    public function update(Request $request)
    {
        $validated = $request->validate([
            'name'  => ['required', 'string', 'max:255'],
            'email' => ['required', 'email',
Rule::unique('users')->ignore($request->user()->id)],
        ]);

        $request->user()->update($validated);
        return back()->with('success', 'Profile updated.');
    }

    public function updatePassword(Request $request)
    {
        $validated = $request->validate([
            'current_password' => ['required', 'current_password'],
            'password'         => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $request->user()->update([
            'password' => Hash::make($validated['password']),
```

```
        ]);

        return back()->with('success', 'Password changed.');
    }
}
```

GOTCHAS & PRO TIPS

- Rule::unique('users')->ignore($id) — essential when letting users update their email, otherwise 'already taken' fires on their own record.
- 'current_password' rule is built-in — verifies the submitted password matches the logged-in user's current one.

MENTAL MODEL

Profile update = two concerns: identity (name, email) and credentials (password). Keeping them in separate forms with separate validators matches how users think about them.

EPISODE 42: DEPLOY AND THEN IMPLEMENT A FEATURE REQUEST

What You'll Learn: Push to production via Forge + DigitalOcean. Forge provisions a server, connects to your GitHub repo, and gives you deploy scripts. Then: add one more feature post-deploy to prove the workflow.

KEY CONCEPTS

- **Laravel Forge** — server provisioning/management service by the Laravel team. $12/mo.
- **DigitalOcean** — cheap VPS host. $6/mo droplet is enough to start.
- Connect Forge to DO via API token. Create a server. Forge installs nginx, PHP, MySQL, Redis, certbot.
- Push to main → Forge pulls, runs composer install, php artisan migrate --force, npm run build.
- Zero-downtime deploys and SSL certs are one-click.

PRACTICAL CODE

Forge's default deploy script — customize in dashboard

```
cd /home/forge/ideas.yourdomain.com
git pull origin main
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
npm ci
npm run build
php artisan queue:restart
```

.env in production — critical flags

```
APP_ENV=production
APP_DEBUG=false              # never leave this true in prod
APP_URL=https://ideas.yourdomain.com
```

```
DB_CONNECTION=mysql          # SQLite is fine dev, MySQL for real prod
QUEUE_CONNECTION=redis
CACHE_STORE=redis
MAIL_MAILER=postmark         # or mailgun, ses, etc.
```

GOTCHAS & PRO TIPS

- APP_DEBUG=true in production leaks secrets (database credentials, API keys) on error pages. Non-negotiable.
- Always --force migrations in prod — Laravel refuses destructive migrations without it.
- composer install uses --no-dev — never install PHPUnit/Pest/Faker on the prod server.
- Use Forge's 'Deployments' panel to watch deploys live. Failed deploy? Roll back one commit and redeploy.

MENTAL MODEL

Forge replaces 'SSH into the server and manually run commands' with 'click a button'. You still need to know what the button does — the script above is yours to own.

EPISODE 43: WHERE TO GO FROM HERE

What You'll Learn: You now know enough Laravel to build real apps. The ecosystem has dozens of specialized tools — here's the rough roadmap of what to learn next and when.

KEY CONCEPTS

- **Livewire** — build dynamic UIs without leaving PHP. SPA-like feel from Blade components.
- **Inertia + Vue/React** — if you want a real SPA frontend but keep Laravel's routing/auth. (This is your stack, Ben!)
- **Filament** — admin panel / CRUD generator. Saves weeks on internal tools.
- **API Resources** — transform Eloquent models into JSON for APIs.
- **Sanctum / Passport** — API auth (tokens vs OAuth).
- **Broadcasting + Reverb** — real-time events (websockets) without a separate Node server.
- **Laravel Horizon** — dashboard for Redis queues in production.
- **Laravel Telescope** — dev debugging dashboard. Every query, job, mail, notification visible.
- **Pint** — opinionated code formatter (PSR-12 by default).
- **Rector** — automated PHP upgrades (8.1 → 8.3 → 8.4).
- **Laravel Pulse** — app health dashboard (slow queries, jobs, usage).

MENTAL MODEL

You don't learn everything at once. Pick the next feature your app needs, then learn the tool that solves it. The docs at laravel.com are the best in any language — use them.

TOOLING REFERENCE

Tools the course uses or recommends. Each gets a one-paragraph 'what and why' so you can decide whether to install it.

LARAVEL HERD

A native Mac/Windows app that bundles PHP 8.4, nginx, and optional DB engines. Any folder you drop in ~/Herd is automatically served at folder-name.test. Zero configuration. This replaces MAMP, Valet, Homebrew-installed PHP, and vhost configs. Install once; forget forever.

PEST

Laravel's default test framework (replaces PHPUnit's ugly syntax with it('does x', fn() => ...)). Pest 4 adds real browser testing with headless Chrome. Use Pest for all new Laravel projects — it's faster to write and reads like English.

PINT

Opinionated PHP code formatter by Laravel. Enforces PSR-12 out of the box. Run with ./vendor/bin/pint before commits or on save. No configuration needed; no arguments about brace placement.

RECTOR

Automated PHP refactoring tool. Upgrading from PHP 8.1 to 8.3? Rector rewrites your code for you — adding readonly, converting to constructor promotion, updating deprecated functions. Essential for maintaining older codebases.

DAISYUI

Tailwind CSS plugin that adds pre-styled component classes (btn, card, modal, input). Additive, not a replacement — you still have full Tailwind available. Skipping it means writing 'py-2 px-4 bg-blue-600 hover:bg-blue-700 text-white rounded' instead of 'btn btn-primary'.

ALPINEJS

~15KB of JavaScript for declarative interactivity. Perfect for toggles, dropdowns, modals, form helpers. Use Alpine when you'd otherwise write 10 lines of vanilla JS; use Livewire or Inertia when you need real data-flow.

VITE

Modern asset bundler (replaces Webpack/Laravel Mix). Dev mode = instant HMR; prod build = optimized and tree-shaken. Laravel ships pre-configured; you rarely touch vite.config.js.

CODE RABBIT

AI-powered code review bot for GitHub PRs. Leaves inline comments on style, bugs, and potential issues. Useful as a second pair of eyes on solo projects. Optional — not course-required.

GLOSSARY

Short definitions of the terms that appear throughout Laravel documentation and this guide. If you see a word you don't recognize, check here first.

Term	Definition
Action Class	A single-purpose invokable class (usually named VerbNounAction) that encapsulates a piece of business logic. Controllers call actions to keep themselves thin. Example: CreateIdeaAction.
Alpine Directive	An x-prefixed HTML attribute that Alpine.js interprets. x-data declares state; x-show toggles visibility; x-on:click (or @click) binds events. Think of them as 'reactive HTML attributes'.
Blade Component	A reusable piece of UI defined in a .blade.php file under resources/views/components/. Used as <x-card :idea="$idea" />. Components accept props and render HTML — like a function, but for templates.
Dependency Injection	The pattern of passing dependencies into a class (via its constructor or method signature) instead of instantiating them internally. Laravel's service container does this automatically when you type-hint a class.
Eager Loading	Pre-fetching related models in a single query with ->with('relation'). Solves the N+1 problem: without eager loading, accessing a relationship inside a loop triggers one query per iteration.
Eloquent	Laravel's ORM (Object-Relational Mapper). Each database table has a Model class; each row is an instance. Queries read like English: User::where('active', true)->get().
Facade	A static-looking proxy for a service container binding. Hash::make(...) is really a call to the Hasher service under the hood. They provide a clean syntax without giving up testability.
Factory	A class (in database/factories/) that generates fake model instances for testing and seeding. Idea::factory()->count(50)->create() makes 50 fake ideas in one line.
Form Request	A dedicated class for validating and authorizing a single HTTP request. Generated by php artisan make:request StoreIdeaRequest. Contains authorize() and rules() methods.
Gate	A named closure that answers a yes/no authorization question. Gate::define('update-idea', fn($user, $idea) => ...). Used via Gate::allows() or @can.
Herd	A native Mac/Windows app that bundles PHP, nginx, and databases — the quickest way to run Laravel locally.

Term	Definition
HMR (Hot Module Reload)	Vite's feature of updating a browser view without a full page reload when you change CSS/JS/Blade files. Preserves form state while iterating on styling.
Mass Assignment	Filling multiple model attributes at once via Model::create([...]) or $model->fill([...]). Laravel requires you to declare $fillable or $guarded to explicitly allow/block attributes — prevents attackers from setting admin=true via form input.
Middleware	A filter that runs before or after a route handler. Auth middleware blocks guests; CSRF middleware rejects forged requests. You can write custom middleware for rate limiting, logging, locale detection, anything.
Migration	A versioned description of a schema change. Each migration has up() and down() methods. php artisan migrate runs pending ones in order; rollback runs down() in reverse.
N+1 Problem	A performance bug where looping over N models triggers N extra queries (one per model) to fetch related data. Fix with eager loading: Idea::with('user')->get().
Policy	A class that groups authorization methods for a specific model. IdeaPolicy has view(), update(), delete() methods that Laravel auto-resolves when you call $this->authorize('update', $idea).
Resource Controller	A controller with all 7 REST methods (index, create, store, show, edit, update, destroy) scaffolded. Generated by php artisan make:controller IdeaController --resource.
Route Model Binding	Laravel's feature of automatically fetching a model from a route parameter. public function show(Idea $idea) — Laravel sees the type hint, looks up the Idea by the {idea} URL parameter, and throws 404 if not found.
Seeder	A class (database/seeders/) that inserts initial data into the DB. Run with php artisan db:seed. Use factories inside seeders for realistic fake data.
Service Container	Laravel's dependency injection engine. When you type-hint a class anywhere (controller method, job, event listener), the container auto-resolves and injects it. Bind custom interfaces to implementations in a service provider.
Vite	The asset bundler Laravel uses for CSS/JS. @vite() directive in your layout injects the right <script>/<link> tags. npm run dev for HMR; npm run build for production.

NEXT STEPS

You've finished the course. You can build real Laravel apps. Here's the roadmap for what to study next — in rough order of what you'll need first.

IMMEDIATE (NEXT 1–2 WEEKS)

- Build a small side project from scratch (not the Ideas app). Pick any domain you care about. Struggle through it without copying. This is where the learning sticks.
- Install Laravel Telescope on that project. Watch every query, job, mail, and notification. Understanding what your app DOES makes you 10x better at debugging.
- Read the Laravel docs — specifically the Eloquent, Validation, and Authorization sections. The course teaches a fraction; the docs are comprehensive.

NEXT 1–2 MONTHS

- Learn Livewire OR Inertia + Vue/React. Pick based on the project. Livewire = 'dynamic Blade', Inertia = 'SPA with Laravel backend'. Your current stack uses Inertia, so start there.
- Learn Filament. It's the fastest way to build admin panels. If you're building any internal tool or CRUD-heavy app, Filament will save you weeks.
- API Resources + Sanctum. Once you need a mobile app or external consumer, you need a real API. Sanctum handles token auth without OAuth complexity.

NEXT 3–6 MONTHS

- Testing mastery. Write browser tests with Pest 4. Use database factories as the bedrock. Aim for every new feature to ship with tests. It's slow at first, fast forever after.
- Broadcasting + Laravel Reverb. Real-time features (live updates, notifications, chat) without standing up a separate Node server. Reverb is Laravel's first-party websocket server.
- Queue architecture. Beyond ShouldQueue — learn about retries, rate limiting, batching, chains, and Horizon for production monitoring.
- Performance tuning. DB indexing, query profiling with Debugbar, cache strategies. Once you have real users, this is where the gains are.

ARCHITECTURE / STAYING-SHARP READING

- 'Laravel Beyond CRUD' (spatie.be) — a free online book on structuring larger Laravel apps with domain-driven design.
- 'Refactoring to Actions' — case studies on extracting logic from fat controllers.
- PHP 8.4 features — property hooks, asymmetric visibility. Jeffrey's OOP course covers these.
- Follow Taylor Otwell, Jeffrey Way, Freek Van der Herten, Caleb Porzio on X/Twitter. They push the ecosystem forward.

CAREER TIP

Build in public. Your LinkedIn (you've already done the overhaul) and a personal site showing 2–3 real Laravel apps beats any certificate. The Laravel community is small and welcoming — contributing a PR to any Spatie or Laravel package is a free credential.

End of guide · Keep shipping · ben@stratusolve.com